

## 기존 Relightable 3DGS 구조와 출발점

R3DG는 일반적인 3D Gaussian Splatting에 **재질과 조명 속성을 추가**하여, relighting이 가능하도록 만든 방법이다.

각 가우시안 속성

$$G_i = \{\mu_i, q_i, s_i, \alpha_i, c_i, n_i, b_i, r_i, l_i\}$$

$\mu_i, q_i, s_i$ : Gaussian의 위치·회전·크기

$\alpha_i$ : 불투명도

$c_i$ : 기존 view-dependent color

$n_i$ : normal

$b_i$ : RGB albedo

$r_i$ : roughness

$l_i$ : Gaussian별 indirect light

R3DG는 두 단계로 학습

→ 첫 번째 단계에서는 Gaussian의 위치·크기·방향과 표면 normal을 학습한다. 이후 각 Gaussian이 어느 방향의 빛을 볼 수 있는지 visibility를 미리 계산한다.

→ 두 번째 단계에서는 geometry를 고정한 상태에서 Gaussian별 RGB albedo, roughness, 직접광, 간접광을 학습한다. 마지막으로 각 Gaussian에서 PBR 색을 계산하고, 기존 3DGS의 alpha blending을 통해 최종 relighting 영상을 만든다.

\*PBR : 표면의 재질, 조명, 관찰 방향을 이용해 빛의 반사 결과를 물리적으로 계산하는 렌더링 방식

R3DG 논문은 정적인 장면만 다룬다는 점, 대규모 장면에서는 Gaussian 수와 PBR 계산량이 증가한다는 점, geometry 정확도를 개선할 필요가 있다는 점을 한계로 언급한다.

-----  
다만 이 논문에서 **RGB 표현 자체의 metamerism 문제를 직접적인 한계로 제시하지 않는다.**

Section 3.3에서 albedo(base color)를

$b_i \in [0, 1]^3$  의 RGB 3채널로 정의하며, Section 3.4의 조명 regularization 역시 R, G, B 채널을 기준으로 계산.

→ **RGB는 여러 파장의 빛을 단 세 값으로 압축한 결과이므로, 서로 다른 실제 재질이 동일한 RGB로 관측될 수 있다.** 따라서 RGB 재구성에는 성공하더라도, 학습하지 않은 spectral illumination 아래에서는 실제 재질과 다른 색 변화가 나타날 수 있다.

**기존 R3DG의 geometry와 alpha splatting 구조는 유지하고, Gaussian별 material representation과 multi-view feature aggregation 부분을 중점적으로 확장한다.**

## 왜 RGB 대신 Spectrum을 사용?

카메라가 기록하는 RGB는 물체의 재질만을 보여주는 값이 아니고, 광원이 어떤 파장의 빛을 내는지, 물체가 각 파장을 얼마나 반사하는지, 카메라가 각 파장에 얼마나 민감한지가 합쳐진 결과

$$I_c = \int E(\lambda)R(\lambda)S_c(\lambda)d\lambda$$

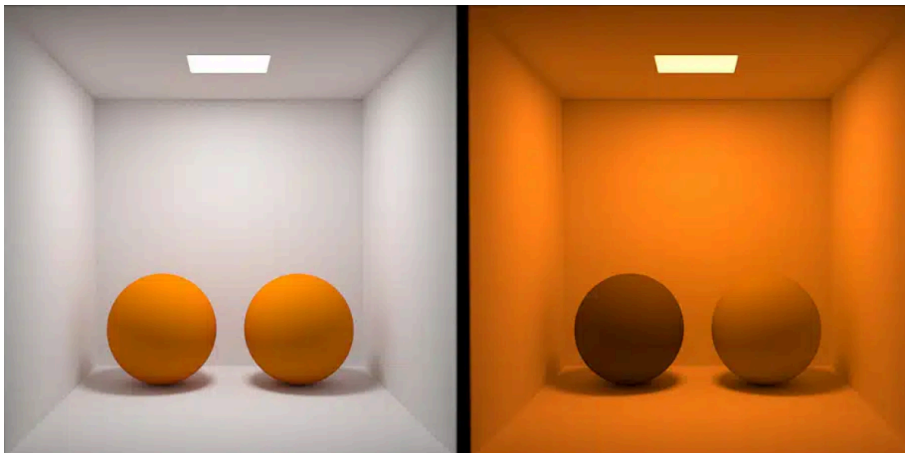
$E(\lambda)$ : 조명 스펙트럼

$R(\lambda)$ : 표면 반사 스펙트럼

$S_c(\lambda)$ : 카메라 감도

$I_c$ : 최종 RGB 관측값

실제 재질이 서로 다른 두 물체라도 특정 조명 아래에서는 사람의 눈이나 카메라에 같은 색으로 보일 수 있다. 하지만 조명이 바뀌면 두 물체의 색이 서로 다르게 변할 수 있다.  
→ 이러한 현상을 **metamerism**이라고 함.



같은 조명에서는 비슷한 색으로 보이던 두 재질이, 다른 조명에서는 서로 다른 색으로 나타나는 예시

제안

$$b_i = [R_i, G_i, B_i] \rightarrow R_i(\lambda)$$

따라서 Gaussian의 재질 표현을

으로 변경

최종적으로는 하나의 스펙트럼을 확정하는 대신 가능한 스펙트럼의 분포를

$$\mathcal{R}_i = \left\{ R_i^{(1)}(\lambda), \dots, R_i^{(M)}(\lambda) \right\}$$

이렇게 표현한다.

→ 각 Gaussian에 하나의 spectrum을 정답으로 고정하지 않고, 현재 RGB 관측과 모순되지 않는 여러 spectral reflectance 후보를 유지한다.

어떤 부분 및 단계에서 변경하는지는 뒤에 이어서

## 착안 ① One-to-Many Spectral Upsampling

이 논문은 기존 RGB-to-spectrum 방법들이 하나의 RGB에 하나의 spectrum만 대응시키는 one-to-one mapping을 사용한다고 지적한다. 그러나 동일한 RGB를 재현하는 spectrum의 집합이 존재하므로, 이를 탐색하는 one-to-many mapping을 제안한다.

가져오는 부분: 동일한 RGB를 재현할 수 있는 여러 spectrum을 함께 다룬다는 개념

새로 설계할 부분: 여러 시점의 특징을 이용해 Gaussian별 후보를 생성하고, 후보의 상대적 가중치를 구하는 방식

## 착안 ② HyperGS: Hyperspectral 3D Gaussian Splatting

HyperGS는 hyperspectral 카메라로 얻은 실제 파장별 데이터를 입력으로 사용하지만, 이 실험은 RGB 영상으로부터 spectral reflectance 후보를 추론하므로 해가 하나로 결정되지 않는 문제이다.

→ HyperGS에서 고차원 스펙트럼을 그대로 Gaussian에 저장하지 않고 저차원 표현으로 압축한다는 방향을 참고하였다. 향후 실험에서는 이를 구현하기 위한 초기 방법으로 spectral basis coefficient를 고려한다. (이것도 추가로 구체적인 설계 필요)

ex. 파장 값을 31개씩 모두 저장하면 너무 무겁다. 그래서 자주 나타나는 기본 스펙트럼 곡선 몇 개를 준비하고, 각 Gaussian이 이 곡선들을 얼마나 섞는지만 저장한다.

$$R(\lambda) = 0.5B_1(\lambda) + 0.3B_2(\lambda) + 0.2B_3(\lambda)$$

spectrum 전체를 저장하는 대신, 기본 곡선 1을 50%, 기본 곡선 2를 30%, 기본 곡선 3을 20% 섞는다고 기록하는 것이다. (가중치st)

왜 **CNN** 특징을 여러 시점에서 **Gaussian**별로 통합하는가?

→ 픽셀 하나의 **RGB**만으로는 재질을 구분하기 어렵다. 그러나 주변의 질감, 경계, 하이라이트 모양과 반복 패턴을 함께 보면 재질과 관련된 단서를 얻을 수 있다. **CNN**은 이러한 주변 정보를 특징 벡터로 압축하는 역할을 한다.

**HSCNN+**는 **RGB** 픽셀 하나만 사용하는 것이 아니라 주변 픽셀의 색과 공간적 패턴을 함께 활용하면 **spectrum** 복원에 도움이 된다는 것을 보여주었다.

**Abstract:**

Hyperspectral recovery from a single RGB image has seen a great improvement with the development of deep convolutional neural networks (CNNs). In this paper, we propose two advanced CNNs for the hyperspectral reconstruction task, collectively called HSCNN+. We first develop a deep residual network named HSCNN-R, which comprises a number of residual blocks. The superior performance of this model comes from the modern architecture and optimization by removing the hand-crafted upsampling in HSCNN. Based on the promising results of HSCNN-R, we propose another distinct architecture that replaces the residual block by the dense block with a novel fusion scheme, leading to a new network named HSCNN-D. This model substantially deepens the network structure for a more accurate solution. Experimental results demonstrate that our proposed models significantly advance the state-of-the-art. In the NTIRE 2018 Spectral Reconstruction Challenge, our entries rank the 1st (HSCNN-D) and 2nd (HSCNN-R) places on both the "Clean" and "Real World" tracks. (Codes are available at [clean-r], [realworld-r], [clean-d], and [realworld-d].)

여기서 참고한 점은

→ **CNN**은 각 이미지에서 재질과 표면에 관련된 특징을 추출한다. 실제 **spectrum** 후보 생성은 여러 시점의 특징을 **Gaussian**별로 모은 이후 별도의 모델이 수행한다.

큰 틀로는

**RGB** → **CNN** 특징 → **Gaussian**별 통합 → **Spectrum** 후보 생성

**Multi-view** 로 한 이유. 뒤에 실험을 진행해보았다!!

한 장의 사진에서는 그림자나 반사 때문에 재질 특징이 왜곡될 수 있다. 같은 표면을 여러 각도에서 보면 어떤 특징은 시점에 따라 바뀌고, 어떤 특징은 반복해서 유지된다. 여러 시점에서 공통으로 나타나는 정보를 모으면 한 장만 사용할 때보다 안정적인 **3D** 재질 특징을 만들 수 있다.

## 실험(부분)

**Q. 동일한 물체의 3D point를 여러 시점에서 관측한 2D CNN feature를 통합하면, 단일 시점 feature보다 시점 변화에 강한 3D feature를 만들 수 있는가?**

### 1. 사용한 데이터

사용 데이터는 NeRF Synthetic Lego입니다. 여러 카메라 시점의 RGB 이미지, 알려진 camera pose, 그리고 `lego.ply` 3D point/Gaussian-like centers를 사용했습니다.



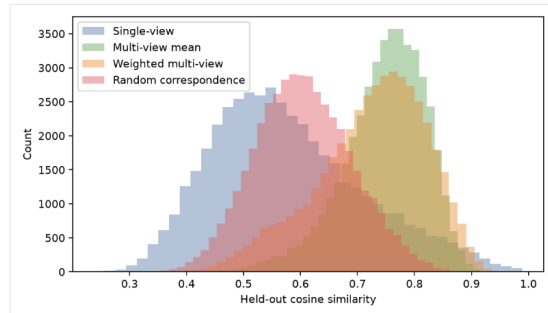
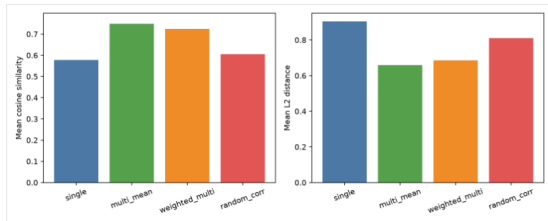
위 이미지는 실제 feature extraction에 사용한 pseudo-RGB 입력 view 일부입니다.

## 2. 무엇을 비교했나

공통 평가 방식은 간단합니다. 어떤 3D point가 여러 이미지에서 보이면, 그중 한 view를 일부러 빼고 held-out 정답처럼 둡니다. 나머지 view들로 만든 feature가 빠진 view의 실제 feature와 얼마나 비슷한지 봅니다.

| 방법                    | 무엇을 하는 방법인가                                                                                                                                               | 왜 비교했나                                                                                                                                                                |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Single-view           | 같은 3D point를 본 이미지 중 한 장의 CNN feature만 사용합니다.                                                                                                             | 가장 단순한 기준선입니다. “한 장만 쓰면 어느 정도인가?”를 보기 위해 넣었습니다.                                                                                                                       |
| Multi-view mean       | 같은 3D point를 본 여러 이미지의 feature를 단순 평균합니다.                                                                                                                 | 이번 가설의 핵심 방법입니다. 여러 view를 모으면 feature가 안정되는지 확인합니다.                                                                                                                   |
| Weighted multi-view   | 여러 view를 그냥 똑같이 평균하지 않고, 더 정면에서 본 view에 조금 더 큰 가중치를 주는 평균입니다. 이번 실험에서는 3D point의 대략적인 radial direction과 camera 방향을 이용한 간단한 viewing-angle heuristic을 썼습니다. | 실제 3DGS/R3DG에서는 모든 view가 똑같이 믿을 만하지 않습니다. 비스듬한 view, 경계 근처 view, 가려진 view는 feature가 덜 정확할 수 있습니다. 그래서 “좋은 view에 더 큰 비중을 주면 단순 평균보다 나아지는가?”를 확인하려고 넣었습니다.              |
| Random correspondence | 일부러 다른 3D point의 feature를 가져와서 비교합니다. 즉, 틀린 3D correspondence를 연결한 가짜 방법입니다.                                                                              | 음성 대조군입니다. 올바른 3D point끼리 모은 feature가 정말 의미가 있다면, 아무 point나 잘못 연결한 random correspondence보다 좋아야 합니다. 이 값이 높으면 ResNet feature가 너무 coarse하거나, 주변 물체/배경이 비슷하다는 뜻일 수 있습니다. |

## 3. 전체 결과 그래프



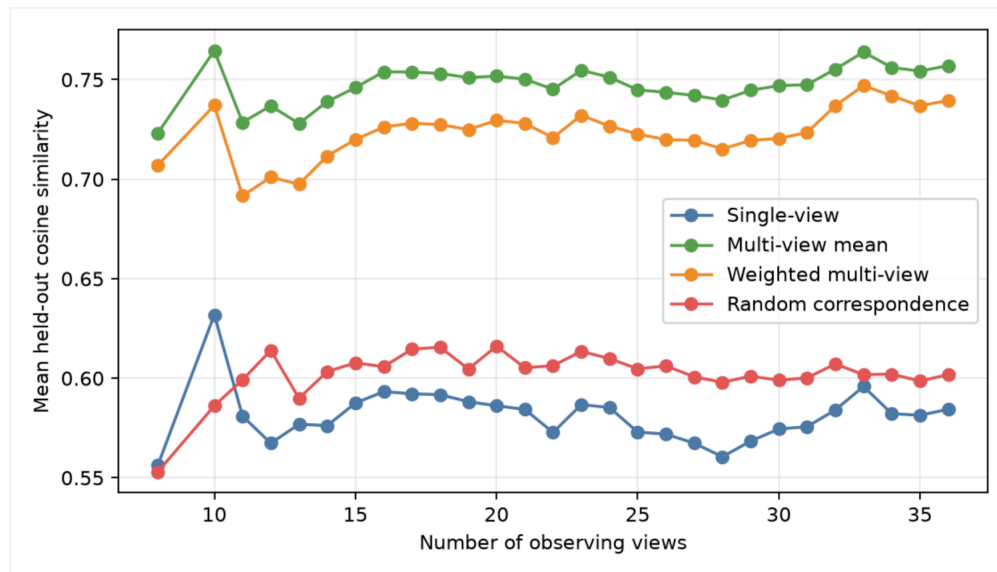
왼쪽 막대그래프의 왼쪽 패널은 cosine similarity입니다. 1에 가까울수록 held-out view feature와 방향이 비슷하다는 뜻이라 좋습니다. 오른쪽 패널은 L2 distance입니다. 작을수록 feature 벡터 사이 거리가 가까워서 좋습니다.

| 방법                    | Cosine similarity ↑ | L2 distance ↓ | 쉽게 읽으면                                               |
|-----------------------|---------------------|---------------|------------------------------------------------------|
| Single-view           | 0.5778              | 0.9047        | 한 장만 쓰면 view 변화에 꽤 흔들립니다.                            |
| Multi-view mean       | 0.7482              | 0.6586        | 가장 좋습니다. 여러 view 평균이 held-out view와 제일 잘 맞았습니다.      |
| Weighted multi-view   | 0.7249              | 0.6847        | single-view보다 좋지만, 단순 평균보다는 조금 낮았습니다.                |
| Random correspondence | 0.6049              | 0.8099        | 틀린 point도 어느 정도 비슷하게 나옵니다. feature가 coarse하다는 경고입니다. |

핵심 결과: Multi-view mean은 single-view보다 cosine이 약 0.1704 높고, L2 distance도 약 0.246 낮습니다. 즉, 같은 3D point에 여러 view feature를 모으는 것이 한 장만 쓰는 것보다 더 안정적이었습니다.

Weighted multi-view가 단순 평균보다 낮았다는 점도 중요합니다. “가중치를 주면 무조건 좋아진다”가 아니라, 어떤 view가 좋은 view인지 판단하는 기준이 정교해야 합니다. 이번 viewing-angle heuristic은 충분히 좋은 visibility/reprojection weight는 아니었습니다.

#### 4. 관측 view 수에 따른 변화



이 그래프는 “같은 3D point가 몇 개의 view에서 관측되었는가”에 따라 성능이 어떻게 달라지는지 보여줍니다. x축은 관측 view 수, y축은 held-out cosine similarity입니다. y값은 높을수록 좋습니다.

이번 실험에서는 평가에 사용된 point들이 최소 4개 이상의 관측 view를 가져야 했고, 실제 그래프에는 주로 8개에서 27개 정도의 관측 view를 가진 point들이 나타났습니다. 전체적으로 초록색 Multi-view mean 선이 파란색 Single-view 선보다 계속 위에 있습니다.

쉽게 말하면, 한 3D point를 여러 방향에서 본 기록이 있을수록 그 기록들을 평균낸 feature가 held-out view와 더 잘 맞았습니다. 특히 single-view는 어떤 한 장을 고르느냐에 따라 흔들리지만, multi-view mean은 여러 관측을 평균내서 그 흔들림을 줄였습니다.

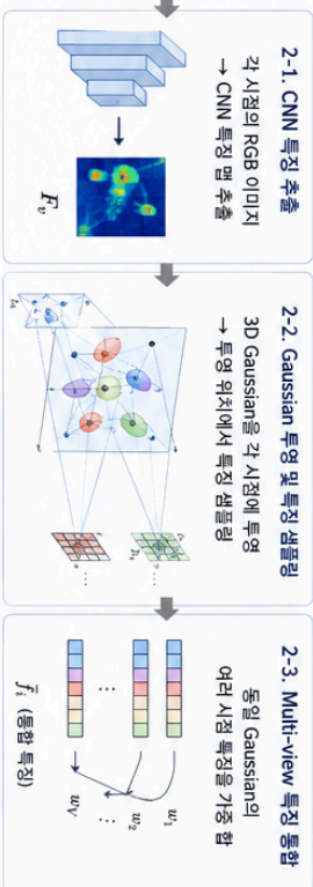
Random correspondence 선이 완전히 낮게 떨어지지 않습니다. Lego 장면에서는 비슷한 노란 부품, 회색 바닥, 반복 구조가 많고, ResNet layer3 feature도 세밀한 픽셀 feature라기보다 넓은 영역의 semantic feature라서 틀린 point도 가끔 비슷하게 보일 수 있습니다.

# 전체 연구 파이프라인: Spectrum 기반 Relightable 3D Gaussian Splatting (제한 방법)

## 1. 입력 (Multi-view RGB & Pose)

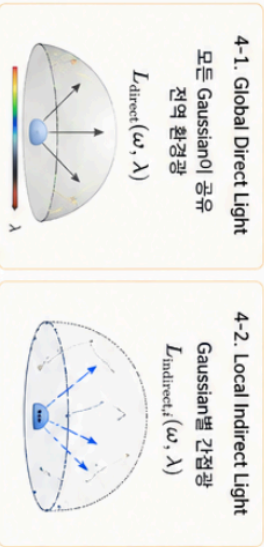


## 2. View-to-Gaussian 특징 추출 및 통합 (현재 검증 완료)

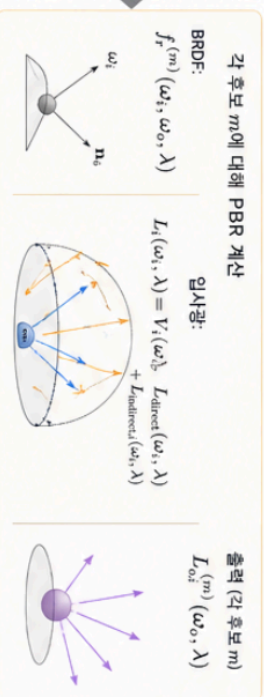


## 3. Gaussian별 복수 Spectrum 후보 추정

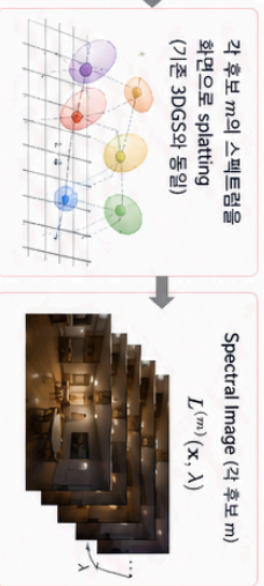
## 4. Spectral Light 추정



## 5. 후보별 Gaussian-level Spectral PBR



## 6. Spectral Alpha Splatting (후보별)



## 7. Camera Response 적분 (스펙트럼 → RGB)



## 8. 최종 출력 (후보 확률 가중 및 분석)



## 학습 제약 (모든 단계에 적용)

- 입력 RGB 재구성 손실
- 물리적 제약:  $0 \leq R(\lambda) \leq 1$
- 스펙트럼 smoothness
- Multi-view 일관성
- 조명 smoothness (공간/방향)
- 후보 다양성 정규화

## 핵심 아이디어

RGB 하나에 여러 가능한 Spectrum 후보를 유지하고, Multi-view CNN 특징과 물리적 제약, 재구성 손실을 함께 활용하여 새로운 조명에서 더 물리적으로 타당한 Relighting을 수행한다.

- 기준 R3DG 구성 유지
- 제한/확장된 부분